

Теория параллелизма

Отчет

Решение уравнения теплопроводности

Выполнил:
Южанинов Максим
Александрович
Группа: 24942

16.05.2026

Цель работы:

Цель работы — реализовать решение уравнения теплопроводности в двумерной области на равномерных сетках методом Якоби с использованием пятиточечного шаблона, выполнить распараллеливание вычислений с помощью OpenACC и сравнить производительность CPU и GPU для различных размеров сетки.

Используемый компилятор:

Для компиляции программы использовался компилятор NVIDIA HPC SDK (`pgc++`) с поддержкой OpenACC. Компиляция выполнялась с ключами: `-acc -Minfo=all`

Ключ `-acc` включает поддержку OpenACC, а `-Minfo=all` выводит информацию о распараллеливании кода.

Использовались режимы:

- 1) `-acc=host` - последовательное выполнение на CPU;
- 2) `-acc=multicore` - выполнение на нескольких ядрах CPU;
- 3) `-acc=gpu` - выполнение на GPU.

Используемый профилировщик:

Для профилирования программы использовался NVIDIA Nsight Systems. Профилировщик применялся для анализа времени выполнения, загрузки GPU и передачи данных между CPU и GPU.

Запуск профилирования: `nsys profile ./a.out`

Как производили замер времени работы:

Время выполнения измерялось средствами библиотеки `std::chrono`. Замер производился перед началом итераций и после их завершения. Тестирование выполнялось для сеток размеров 128×128 , 256×256 , 512×512 и 1024×1024 в режимах CPU, multicore и GPU.

Выполнение на CPU

CPU-onecore

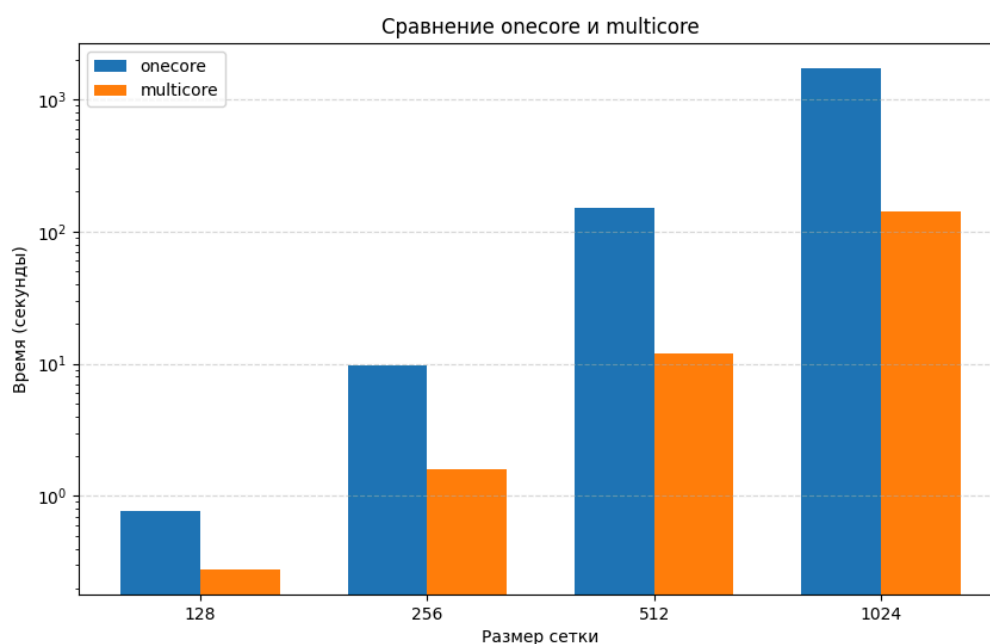
Размер сетки	Время выполнения (секунды)	Точность	Количество итераций
128*128	0.7741	10^{-6}	30074
256*256	9.6234	10^{-6}	102885
512*512	150.2511	10^{-6}	339599
1024*1024	1717.5606	10^{-6}	1000000

CPU-multicore (8 потоков)

Размер сетки	Время выполнения (секунды)	Точность	Количество итераций
128*128	0.2781	10^{-6}	30074
256*256	1.5807	10^{-6}	102885
512*512	12.0323	10^{-6}	339599
1024*1024	142.0009	10^{-6}	1000000

Количество потоков	Время выполнения (секунды)	Точность	Количество итераций	Размер сетки
2	0.2337	10^{-6}	30074	128*128
4	0.1675	10^{-6}	30074	128*128
8	0.2781	10^{-6}	30074	128*128
2	2.7230	10^{-6}	102885	256*256
4	1.5969	10^{-6}	102885	256*256
8	1.5807	10^{-6}	102885	256*256
2	43.1119	10^{-6}	339599	512*512
4	21.0279	10^{-6}	339599	512*512
8	12.0323	10^{-6}	339599	512*512
2	689,8260	10^{-6}	1000000	1024*1024
4	406.0731	10^{-6}	1000000	1024*1024
8	142.0009	10^{-6}	1000000	1024*1024

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарий (что было сделано)
1	233.4566	10^{-6}	1000000	Без изменений
2	24.3652	10^{-6}	1000000	Устранены лишние операции копирования данных
3	22.4765	10^{-6}	1000000	Заменены fabs и fmax на простое условное сравнение
4	26.4790	10^{-6}	1000000	Убраны лишние обращения к памяти внутри ядра
5	26.9051	10^{-6}	1000000	Блочная обработка

Этап 1

```

** CUDA API Summary (cuda_api_sum):

Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
78.4 108331187 6001 18852.2 2705.0 1339 2395516 145166.1 cuStreamSynchronize
9.2 12785235 1000 12785.2 12115.5 11514 41373 2616.5 cuMemcpyDtoHAsync_v2
8.1 11189066 3000 3729.7 3551.5 2726 24957 1150.6 cuLaunchKernel
1.9 2573401 1000 2573.4 2387.0 2125 13031 770.7 cuMemsetD32Async
0.8 1050286 1 1050286.0 1050286.0 1050286 1050286 0.0 cuMemAllocHost_v2
0.7 1035334 5 207066.8 287968.0 4408 378188 185807.0 cuMemAlloc_v2
0.7 1026825 2 513412.5 513412.5 505702 521123 10904.3 cuMemcpyHtoDAsync_v2
0.2 240845 1 240845.0 240845.0 240845 240845 0.0 cuModuleLoadDataEx
0.0 3956 3 1318.7 1318.0 508 2130 811.0 cuCtxSetCurrent

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_kern_sum.py]...

** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
66.1 7321126 2000 3660.6 4784.5 1951 9951 1680.5 calcNext_53(double *, double *, int, int)
33.9 3754274 1000 3754.3 3552.0 3487 9728 867.7 swap_69(double *, double *, int, int)

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_time_sum.py]...

** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):

Time (%) Total Time (ns) Count Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Operation
-----
42.8 1508564 1000 1508.6 1440.0 1407 6144 453.9 [CUDA memcpy Device-to-Host]
37.8 1332631 1000 1332.6 1248.0 1216 12992 635.3 [CUDA memset]
19.4 682394 2 341197.0 341197.0 334301 348093 9752.4 [CUDA MemOp Host-to-Device]

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_size_sum.py]...

** CUDA GPU MemOps Summary (by Size) (cuda_gpu_mem_size_sum):

Total (MB) Count Avg (MB) Med (MB) Min (MB) Max (MB) StdDev (MB) Operation
-----
4.194 2 2.097 2.097 2.097 2.097 0.000 [CUDA memcpy Host-to-Device]
0.008 1000 0.000 0.000 0.000 0.000 0.000 [CUDA memcpy Device-to-Host]
0.008 1000 0.000 0.000 0.000 0.000 0.000 [CUDA memset]

```

Этап 2

```

** CUDA API Summary (cuda_api_sum):

Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
84.7 2649908919 77054 34390.3 3399.5 1363 5332860 248549.2 cuStreamSynchronize
8.2 257050135 19264 13343.5 12631.0 11580 424997 4027.5 cuMemcpyDtoHAsync_v2
5.2 164214907 38526 4262.4 3948.0 2860 854549 5173.5 cuLaunchKernel
1.7 54184116 19263 2812.9 2522.0 2132 437652 3874.2 cuMemsetD32Async
0.1 2023980 1 2023980.0 2023980.0 2023980 2023980 0.0 cuModuleLoadDataEx
0.0 1220678 1 1220678.0 1220678.0 1220678 1220678 0.0 cuMemAllocHost_v2
0.0 257831 5 51566.2 4855.0 3745 223827 96554.7 cuMemAlloc_v2
0.0 66331 2 33165.5 33165.5 27070 39261 8620.3 cuMemcpyHtoDAsync_v2
0.0 2423 3 807.7 1000.0 255 1168 485.9 cuCtxSetCurrent

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_kern_sum.py]...

** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
100.0 79972802 38526 2075.8 2079.0 1759 16032 651.8 calcNext_53(double *, double *, int, int)

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_time_sum.py]...

** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):

Time (%) Total Time (ns) Count Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Operation
-----
52.7 29134735 19264 1512.4 1440.0 1376 12192 557.1 [CUDA memcpy Device-to-Host]
47.3 26118270 19263 1355.9 1248.0 1215 10944 632.3 [CUDA memset]
0.0 20480 2 10240.0 10240.0 9952 10528 407.3 [CUDA memcpy Host-to-Device]

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_size_sum.py]...

** CUDA GPU MemOps Summary (by Size) (cuda_gpu_mem_size_sum):

Total (MB) Count Avg (MB) Med (MB) Min (MB) Max (MB) StdDev (MB) Operation
-----
0.234 19264 0.000 0.000 0.000 0.000 0.001 [CUDA memcpy Device-to-Host]
0.160 2 0.000 0.000 0.000 0.000 0.000 [CUDA memcpy Host-to-Device]
0.154 19263 0.000 0.000 0.000 0.000 0.000 [CUDA memset]

```

Этап 3

```

** CUDA API Summary (cuda_api_sum):

Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
84.6 2800370605 77054 36459.8 2185.0 1371 8052595 257215.2 cuStreamSynchronize
8.1 267984413 19264 13911.2 12911.0 11533 6051824 43840.4 cuMemcpyDtoHAsync_v2
5.4 179261143 38526 4653.0 4327.5 2779 434469 2916.1 cuLaunchKernel
1.9 63713185 19263 3307.5 2970.0 2163 856879 7264.0 cuMemsetD32Async
0.0 1128620 1 1128620.0 1128620.0 1128620 1128620 0.0 cuMemAllocHost_v2
0.0 287591 5 57518.2 4853.0 3735 253898 109998.9 cuMemAlloc_v2
0.0 215774 1 215774.0 215774.0 215774 215774 0.0 cuModuleLoadDataEx
0.0 66704 2 33352.0 33352.0 27916 38788 7687.7 cuMemcpyHtoDAsync_v2
0.0 2817 3 939.0 1257.0 232 1328 613.3 cuCtxSetCurrent

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_kern_sum.py]...

** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
100.0 79848140 38526 2072.6 2080.0 1759 13632 570.7 calcNext_53(double *, double *, int, int)

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_time_sum.py]...

** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):

Time (%) Total Time (ns) Count Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Operation
-----
52.7 29094156 19264 1510.3 1440.0 1407 11808 533.9 [CUDA memcpy Device-to-Host]
47.2 26072296 19263 1353.5 1248.0 1215 18144 635.1 [CUDA memset]
0.0 19200 2 9600.0 9600.0 9536 9664 90.5 [CUDA memcpy Host-to-Device]

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_size_sum.py]...

** CUDA GPU MemOps Summary (by Size) (cuda_gpu_mem_size_sum):

Total (MB) Count Avg (MB) Med (MB) Min (MB) Max (MB) StdDev (MB) Operation
-----
0.234 19264 0.000 0.000 0.000 0.000 0.001 [CUDA memcpy Device-to-Host]
0.160 2 0.000 0.000 0.000 0.000 0.000 [CUDA memcpy Host-to-Device]
0.154 19263 0.000 0.000 0.000 0.000 0.000 [CUDA memset]

```

Этап 4

```

** CUDA API Summary (cuda_api_sum):

Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
82.1 2550175372 77054 33096.0 2555.5 1387 5908992 233291.9 cuStreamSynchronize
8.8 274630474 19264 14256.2 13530.0 11955 739357 6503.5 cuMemcpyDtoHAsync_v2
6.8 210542808 38526 5465.0 5009.0 2953 1018688 6275.9 cuLaunchKernel
2.2 68706025 19263 3566.7 3197.0 2135 438321 4847.8 cuMemsetD32Async
0.0 1226668 1 1226668.0 1226668.0 1226668 1226668 0.0 cuMemAllocHost_v2
0.0 264634 5 52926.8 6774.0 5225 226375 97177.5 cuMemAlloc_v2
0.0 214893 1 214893.0 214893.0 214893 214893 0.0 cuModuleLoadDataEx
0.0 71858 2 35929.0 35929.0 29951 41907 8454.2 cuMemcpyHtoDAsync_v2
0.0 3907 3 1302.3 1725.0 260 1922 908.0 cuCtxSetCurrent

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_kern_sum.py]...

** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
-----
100.0 89129364 38526 2313.5 2560.0 1759 13536 659.2 calcNext_53(double *, double *, int, int)

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_time_sum.py]...

** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):

Time (%) Total Time (ns) Count Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Operation
-----
52.8 29043583 19264 1507.7 1440.0 1407 11776 511.6 [CUDA memcpy Device-to-Host]
47.2 25980501 19263 1348.7 1248.0 1184 13152 602.4 [CUDA memset]
0.0 20160 2 10080.0 10080.0 9792 10368 407.3 [CUDA memcpy Host-to-Device]

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_size_sum.py]...

** CUDA GPU MemOps Summary (by Size) (cuda_gpu_mem_size_sum):

Total (MB) Count Avg (MB) Med (MB) Min (MB) Max (MB) StdDev (MB) Operation
-----
0.234 19264 0.000 0.000 0.000 0.000 0.001 [CUDA memcpy Device-to-Host]
0.160 2 0.000 0.000 0.000 0.000 0.000 [CUDA memcpy Host-to-Device]
0.154 19263 0.000 0.000 0.000 0.000 0.000 [CUDA memset]

```

Этап 5

```
** CUDA API Summary (cuda_api_sum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
95.9	11321706782	77054	146932.1	28392.0	1328	5589613	343928.2	cuStreamSynchronize
2.3	269434879	19264	13986.4	13092.0	12012	809430	8281.4	cuMemcpyDtoHAsync_v2
1.4	162260666	38526	4211.7	3869.0	2875	191944	2043.8	cuLaunchKernel
0.4	52223304	19263	2711.1	2459.0	2102	405515	3055.1	cuMemsetD32Async
0.0	1424900	1	1424900.0	1424900.0	1424900	1424900	0.0	cuModuleLoadDataEx
0.0	1311671	1	1311671.0	1311671.0	1311671	1311671	0.0	cuMemAllocHost_v2
0.0	312023	5	62404.6	9513.0	6243	267661	114921.1	cuMemAlloc_v2
0.0	77623	2	38811.5	38811.5	30104	47519	12314.3	cuMemcpyHtoDAsync_v2
0.0	3281	3	1093.7	1295.0	346	1640	670.1	cuCtxSetCurrent

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_kern_sum.py]...

```
** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
100.0	153827113	38526	3992.8	4480.0	2943	13632	858.3	calcNext_56(double *, double *, int, int)

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_time_sum.py]...

```
** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):
```

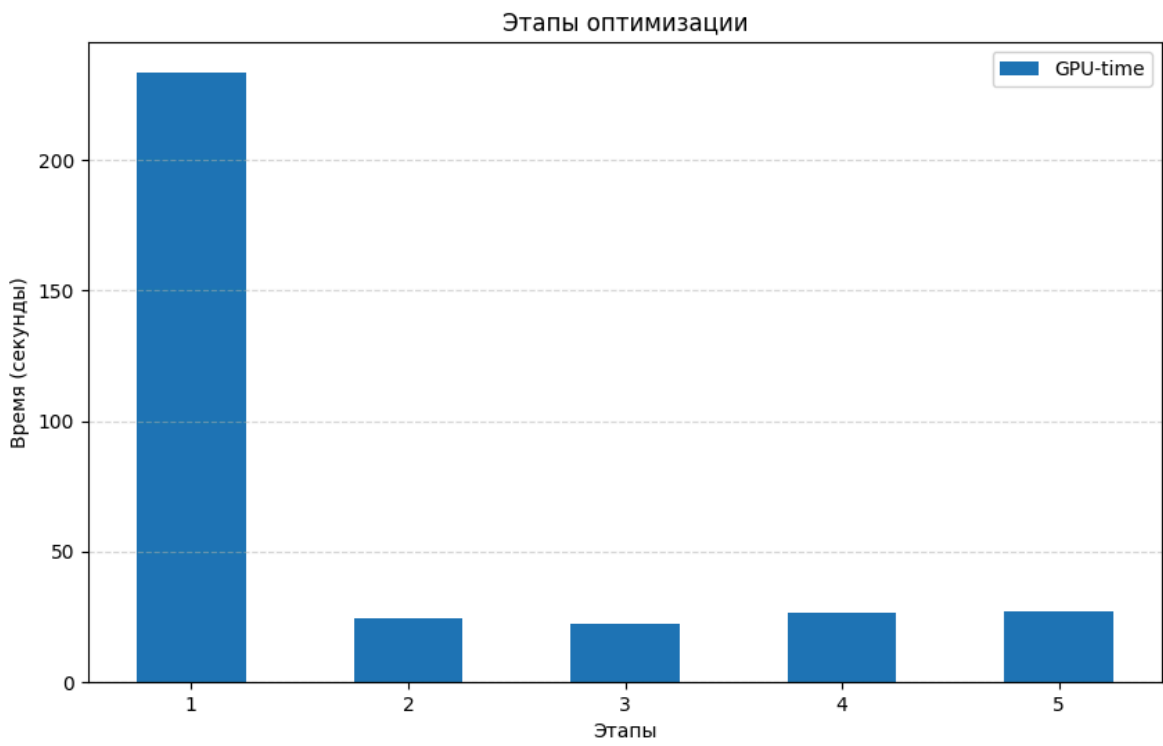
Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
50.7	30629909	19264	1590.0	1536.0	1376	12384	522.7	[CUDA memcpy Device-to-Host]
49.3	29775828	19263	1545.8	1344.0	1216	13792	674.1	[CUDA memset]
0.0	28864	2	14432.0	14432.0	14304	14560	181.0	[CUDA memcpy Host-to-Device]

Processing [report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/23.11/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gpu_mem_size_sum.py]...

```
** CUDA GPU MemOps Summary (by Size) (cuda_gpu_mem_size_sum):
```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
0.234	19264	0.000	0.000	0.000	0.000	0.001	[CUDA memcpy Device-to-Host]
0.160	2	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Host-to-Device]
0.154	19263	0.000	0.000	0.000	0.000	0.000	[CUDA memset]

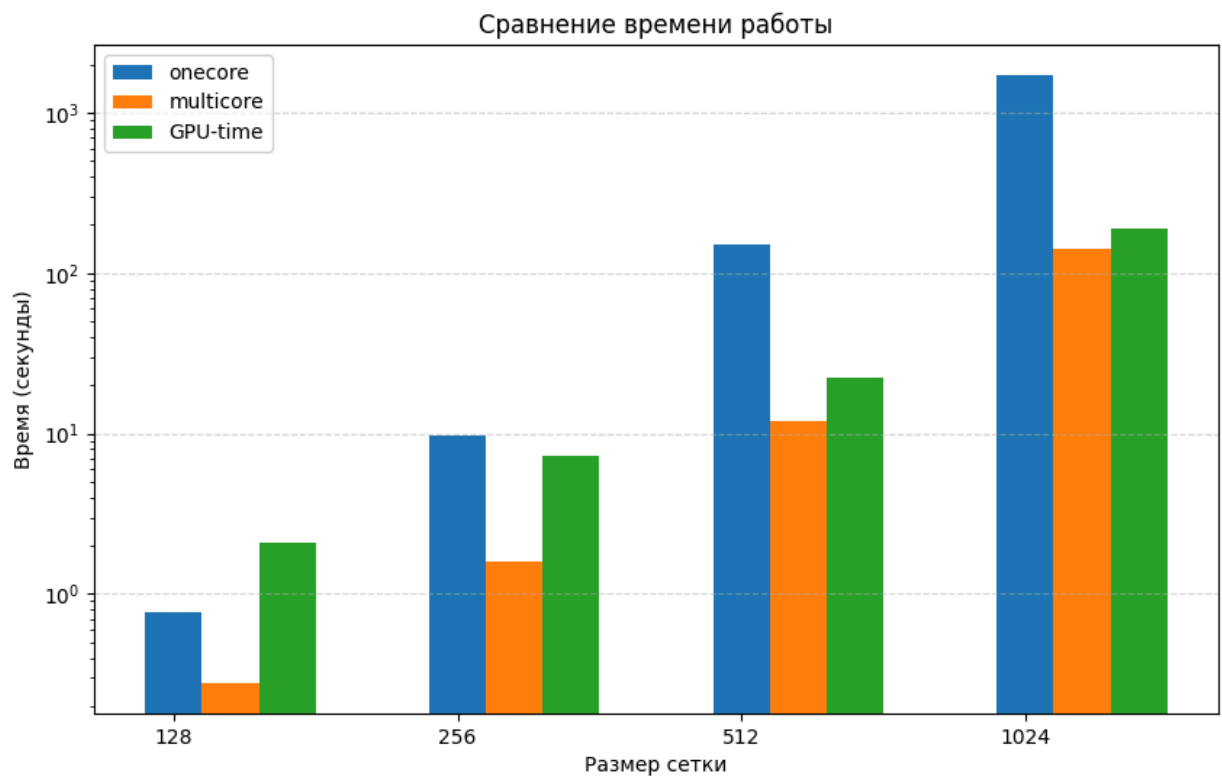
Диаграмма оптимизации



GPU – оптимизированный вариант

Размер сетки	Время выполнения (секунды)	Точность	Количество итераций
128*128	2.0854	10^{-6}	30074
256*256	7.3179	10^{-6}	102885
512*512	22.4765	10^{-6}	339599
1024*1024	191.7050	10^{-6}	1000000

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Вывод:

По полученным результатам можно сделать вывод, что эффективность метода решения уравнения теплопроводности существенно зависит как от размера расчетной сетки, так и от используемой архитектуры вычислений. Во всех экспериментах использовалась одинаковая точность вычислений, а количество итераций определялось сходимостью метода Якоби. Для сеток 128×128 , 256×256 и 512×512 алгоритм достигал сходимости, тогда как для размера 1024×1024 был достигнут установленный предел в 1000000 итераций, что указывает на крайне медленную сходимость метода при больших размерах области.

Для однопоточной CPU-реализации наблюдается резкий рост времени вычислений с увеличением размера сетки. При размере 128×128 время выполнения составило всего 0.7741 с, однако уже для сетки 512×512 оно увеличилось до 150.2511 с, а для 1024×1024 достигло 1717.5606 с. Это связано с квадратичным ростом числа узлов сетки и большим количеством итераций, необходимых для достижения заданной точности.

Многопоточная CPU-реализация с использованием 8 потоков показала значительное ускорение по сравнению с однопоточным вариантом. Для сетки 128×128 время выполнения сократилось до 0.2781 с, что соответствует ускорению примерно в 2.8 раза. Для сетки 512×512 ускорение уже составило около 12.5 раза относительно однопоточной версии. Наиболее заметный эффект распараллеливания наблюдается на больших размерах сетки, поскольку вычислительная нагрузка распределяется между потоками более эффективно, а накладные расходы на создание и синхронизацию потоков становятся менее значимыми.

GPU-реализация продемонстрировала смешанные результаты. На небольших сетках (128×128 и 256×256) графический процессор уступает многопоточной CPU-версии, поскольку время передачи данных и запуска вычислительных ядер превышает выигрыш от массового параллелизма. Однако при увеличении размера сетки преимущество GPU становится более заметным. Для сетки 512×512 время выполнения составило 22.4765 с, что значительно быстрее

однопоточной реализации, но всё ещё медленнее многопоточной CPU-версии. Для размера 1024×1024 GPU выполнил вычисления за 191.7050 с, тогда как однопоточная CPU-версия потребовала 1717.5606 с. Таким образом, ускорение относительно однопоточного варианта составило почти 9 раз.

Полученные результаты показывают, что использование GPU наиболее эффективно для задач большого объёма, где высокая степень параллелизма позволяет компенсировать накладные расходы, связанные с передачей данных и запуском вычислительных ядер. В то же время для задач малого и среднего размера многопоточная CPU-реализация может оказаться более предпочтительной за счёт меньших накладных расходов и более эффективной работы с памятью. Также результаты демонстрируют ограничение метода Якоби: при увеличении размера сетки число итераций резко возрастает, что приводит к значительному увеличению времени вычислений независимо от используемой аппаратной платформы.